AEGISAI ENTERPRISE EDITION

TECHNICAL DOCUMENTATION SUITE

Step-by-Step Google Cloud Deployment Guide

Zero-to-Hero Cloud Run, Artifact Registry & Secret Manager Procedure

Platform Version:	**2.5.0 Enterprise Edition**
Compliance Scope:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
License:	**Apache License 2.0 (Open Source)**

AegisAI Engineering Team

Confidentiality: Public / Open Source

Step-by-Step Google Cloud Installation and Deployment Guide

AegisAI — Enterprise AI Security & Governance Assurance Platform

Version: 2.5.0 Enterprise (2026 Edition)

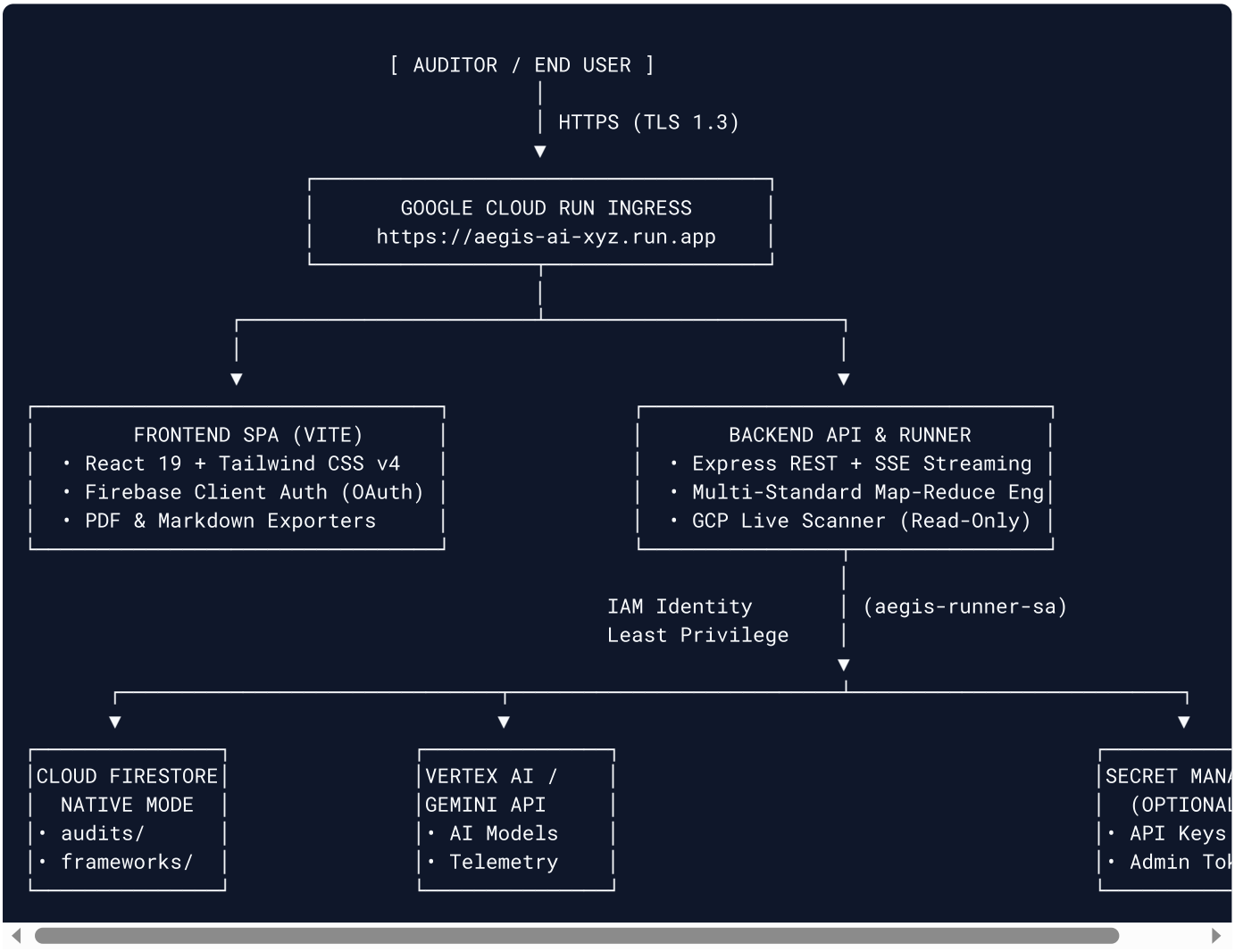
Classification: Official Technical Cloud Infrastructure Deployment Guide

Deployment Platform: Google Cloud Platform (Cloud Run, Native Firestore, Vertex AI, Artifact Registry, Firebase Auth)

Estimated Time: 10 to 15 minutes

1. Google Cloud Deployment Architecture

AegisAI is deployed under a **Serverless Cloud-Native** architecture on Google Cloud Platform. This architecture ensures **high availability (99.95%)**, automatic scaling from 0 to N instances, cost efficiency (pay-per-use), and strict security isolation:



2. Prerequisites

Before getting started, make sure you have:

1. **Google Cloud Account and Project**:
   - An active GCP project with a linked **Cloud Billing Account**.
   - If you do not have one, you can create it in the [Google Cloud Console](https://console.cloud.google.com/).
2. **Required IAM Permissions**:
   - `roles/owner` role **or** the combination of:
   - Project IAM Admin (`roles/resourcemanager.projectIamAdmin`)
   - Cloud Run Admin (`roles/run.admin`)
   - Artifact Registry Admin (`roles/artifactregistry.admin`)
   - Cloud Build Editor (`roles/cloudbuild.builds.editor`)
   - Datastore / Firestore Owner (`roles/datastore.owner`)
   - Service Account Admin (`roles/iam.serviceAccountAdmin`)
3. **Recommended Execution Environment: Google Cloud Shell**:
   - It is recommended to run this deployment directly from **[Google Cloud Shell](https://shell.cloud.google.com)**.
   - Cloud Shell comes with `gcloud` , `docker` , `git` , `node` , and `npm` pre-installed and authenticated with your Google credentials.
4. **Google Gemini API Key (Optional if using Vertex AI directly)**:
   - You can obtain a free or enterprise key in [Google AI Studio](https://aistudio.google.com/).

---

3. Step-by-Step Manual Deployment Guide

Follow these 12 sequential steps in the Google Cloud Shell terminal or your local terminal with authenticated `gcloud` .

Step 1: Configure Environment Variables and Project in `gcloud`

Define the deployment variables and set the active project:

```
# 1. Define your Google Cloud project ID
export PROJECT_ID="your-gcp-project-id"

# 2. Define the region (us-central1 recommended for Vertex AI and Cloud Run)
export REGION="us-central1"

# 3. Define service and repository names
export SERVICE_NAME="aegis-ai"
export REPO_NAME="cloud-run-source-deploy"

# 4. Authorized domains for sign-in (* to allow any Google account)
export ALLOWED_DOMAINS="*"

# 5. (Optional) Gemini API key if you have it handy
export GEMINI_API_KEY="AIzaSyYourGeminiKeyHere"

# 6. Configure the context in gcloud
gcloud config set project "$PROJECT_ID"
```

> [!TIP]

> If you do not know your exact project ID, run `gcloud projects list` to view it.

Step 2: Enable Required Google Cloud APIs

AegisAI requires 8 managed Google Cloud services. Enable them with a single command:

```
echo "Enabling required Google Cloud APIs..."
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  aiplatform.googleapis.com \
  firestore.googleapis.com \
  firebase.googleapis.com \
  identitytoolkit.googleapis.com \
  secretmanager.googleapis.com \
  --project="$PROJECT_ID"
```

Verification: The terminal will confirm `Operation "operations/..." finished successfully`.

Step 3: Configure IAM Permissions for the Cloud Build Pipeline

To allow Google Cloud Build to build and push images to Artifact Registry without permission blocks:

```
# Get the project number
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format="value(projectNumber)")
COMPUTE_SA="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"
CLOUDBUILD_SA="${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"

echo "Configuring permissions for Project #${PROJECT_NUMBER}..."

# Grant roles to the Compute service account
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/cloudbuild.builds.builder" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/storage.admin" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/logging.logWriter" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/artifactregistry.admin" --quiet

# Grant roles to the Cloud Build service account
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${CLOUDBUILD_SA}" \
  --role="roles/cloudbuild.builds.builder" --quiet >/dev/null 2>&1 || true

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${CLOUDBUILD_SA}" \
  --role="roles/storage.admin" --quiet >/dev/null 2>&1 || true
```

Step 4: Create the Least-Privilege Execution Service Account (`aegis-runner-sa` — `C05.2`)

Following **Strict Least Privilege** best practices (`OWASP AI-SVS C05.2`), we create a dedicated identity for Cloud Run granted **only** `roles/aiplatform.user` and `roles/datastore.user` (never granting project-wide `roles/viewer`, `roles/editor`, or `roles/owner` to the runtime container):

```
SA_NAME="aegis-runner-sa"
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"

# 1. Create the service account if it does not exist
if ! gcloud iam service-accounts describe "${SA_EMAIL}" --project="$PROJECT_ID" &>/dev/null; then
    gcloud iam service-accounts create "${SA_NAME}" \
      --description="Least-privilege execution identity for AegisAI Cloud Run" \
      --display-name="AegisAI Runner" \
      --project="$PROJECT_ID"
    echo "Service account created: ${SA_EMAIL}"
else
    echo "Service account already exists: ${SA_EMAIL}"
fi

# 2. Assign Vertex AI Inference role
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/aiplatform.user" --quiet

# 3. Assign Cloud Firestore access role
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/datastore.user" --quiet
```

Step 5: Provision Cloud Firestore in Native Mode

AegisAI stores immutable audit reports and custom regulatory frameworks in Firestore:

```
# Check if the aegis-ai-db database already exists; if not, create it
if ! gcloud firestore databases list --project="$PROJECT_ID" --format="value(name)" 2>/dev/null | gr
    echo "Creating Firestore database aegis-ai-db in Native mode..."
    gcloud firestore databases create \
      --database="aegis-ai-db" \
      --location="nam5" \
      --type="firestore-native" \
      --project="$PROJECT_ID" --quiet
else
    echo "Firestore database already initialized and ready."
fi
```

> [!NOTE]
>
> `nam5` is the United States multi-region (high geographic availability). If you prefer Europe, use `eur3`.

Step 6: Configure Firebase Authentication and Google Sign-In

To allow auditors and compliance officers to sign in via Google:

1. Open the Firebase console: **https://console.firebase.google.com/project/_/authentication/providers**.
2. Select your project (`$PROJECT_ID`).

3. Under the "**Sign-in method**" tab:
   - Click on the "**Google**" provider.
   - Toggle the enable switch.
   - Select your project support email and click **Save**.
4. Under **Project Settings > General**, scroll down to the "**Your apps**" section:
   - If you do not have a registered Web app, click the Web icon ( ), name it `AegisAI`, and click **Register app**.
5. Ensure that the `firebase-applet-config.json` file in the root of your repository contains your project values:

```
{
  "projectId": "your-gcp-project-id",
  "appId": "1:123456789012:web:abcdef1234567890",
  "apiKey": "AIzaSyYourFirebaseApiKey",
  "authDomain": "your-gcp-project-id.firebaseapp.com",
  "firestoreDatabaseId": "aegis-ai-db",
  "storageBucket": "your-gcp-project-id.firebasestorage.app",
  "messagingSenderId": "123456789012"
}
```

> [!TIP]
>
> If you use the automated script `./deploy_cloud_run.sh`, this file is automatically updated with your `PROJECT_ID` and `PROJECT_NUMBER`.

Step 7: Create an Artifact Registry Repository

Artifact Registry will host the compiled Docker container image:

```
if ! gcloud artifacts repositories describe "$REPO_NAME" --location="$REGION" --project="$PROJECT_ID
    echo "Creating Artifact Registry repository..."
    gcloud artifacts repositories create "$REPO_NAME" \
      --repository-format=docker \
      --location="$REGION" \
      --description="Docker repository for AegisAI" \
      --project="$PROJECT_ID" --quiet
else
    echo "Repository ${REPO_NAME} already exists in Artifact Registry."
fi
```

Step 8: Build the Container Image

Define the full image URL:

```
IMAGE_URL="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/${SERVICE_NAME}:latest"
echo "Image destination: $IMAGE_URL"
```

Option A: Cloud Build with Google Cloud (Recommended)

Does not require Docker installed locally:

```
gcloud builds submit --tag "$IMAGE_URL" --project="$PROJECT_ID" .
```

Option B: Local Build with Docker

If you have Docker running locally:

```
docker build --no-cache -t "$IMAGE_URL" .
docker push "$IMAGE_URL"
```

Step 9: Deploy the Service to Google Cloud Run with Zero-Trust IAP & VPC Egress (`C04.3`, `C05.3`)

Deploy the container to Cloud Run behind **Cloud Identity-Aware Proxy** (`--no-allow-unauthenticated` + `--iap`), mounting `ADMIN_API_TOKEN` from Secret Manager and routing private egress through a Serverless VPC Access Connector (`--vpc-egress private-ranges-only`):

```
gcloud beta run deploy "$SERVICE_NAME" \
  --image "$IMAGE_URL" \
  --project "$PROJECT_ID" \
  --region "$REGION" \
  --platform managed \
  --service-account "${SA_EMAIL}" \
  --min-instances 0 \
  --max-instances 10 \
  --memory 1Gi \
  --cpu 1 \
  --timeout 300s \
  --port 8080 \
  --set-env-vars "NODE_ENV=production,PORT=8080,PROJECT_ID=${PROJECT_ID},GOOGLE_CLOUD_LOCATION=${REC
  --set-secrets "ADMIN_API_TOKEN=aegis-admin-token:latest" \
  --vpc-connector "aegis-vpc-connector" \
  --vpc-egress "private-ranges-only" \
  --no-allow-unauthenticated \
  --iap
```

> [!NOTE]

> Enforcing `--no-allow-unauthenticated` and `--iap` satisfies organizational Domain Restricted Sharing policies (`constraints/iam.allowedPolicyMemberDomains`) and OWASP AI-SVS `C04.3` perimeter isolation. Grant `roles/iap.httpsResourceAccessor` and `roles/run.invoker` only to your authorized corporate domains (`domain:google.com` , `domain:altostrat.com`).

Step 10: Retrieve the Assigned URL and Authorize the Domain in Firebase

Once the deployment is complete, retrieve the public URL of your service:

```
SERVICE_URL=$(gcloud run services describe "$SERVICE_NAME" --project "$PROJECT_ID" --region "$REGION
echo "============================================================"
echo "🎉 AEGIS-AI PUBLIC URL: $SERVICE_URL"
echo "============================================================"
```

Mandatory Step for Google OAuth:

1. Copy the Cloud Run domain without `https://` (e.g., `aegis-ai-607603788049.us-central1.run.app`).
2. Open: **[Firebase Console > Authentication > Settings > Authorized domains] (https://console.firebase.google.com/project/_/authentication/settings)**.
3. Click on "**Add domain**".
4. Paste the domain and click **Save**.

Step 11: Health Verification (Smoke Tests)

Verify that the platform is responding correctly:

```
# 1. Check health endpoint
curl -s "${SERVICE_URL}/api/health"
# Expected response: {"status":"ok"}

# 2. Check HTTP security headers
curl -I "${SERVICE_URL}"
# Should include: X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN
```

---

4. Alternative Method: Automated 1-Step Deployment

AegisAI includes the orchestration script `deploy_cloud_run.sh`, which automatically executes all the previous steps (1 through 10):

```
# 1. Grant execution permissions
chmod +x deploy_cloud_run.sh

# 2. Run the script by passing your GCP project ID
./deploy_cloud_run.sh YOUR_GCP_PROJECT_ID
```

The script will validate dependencies, enable the APIs, create service accounts, build the container, and deploy to Cloud Run, finally printing the ready-to-use URL.

---

5. Post-Deployment Configuration

5.1 Configuring AI Keys from the Web Interface

1. Open your service URL in a browser.
2. Log in with your Google account (or use the "**Demo / Sandbox Mode**" button).
3. Navigate to the **Settings / Administration** view (gear icon ⚙️ in the side menu).

4. Under the **Google Gemini** tab, enter your API Key if you did not configure it during deployment.
5. Click "**Test Connection**" to verify latency and the active model.

5.2 Multi-Provider Configuration (Optional)

If your company prefers to route requests through **OpenRouter**, **OpenAI**, or an OpenAI-compatible local server (**vLLM**, **Ollama**, **LiteLLM**):

- Select the provider in the corresponding Administration tab.
- Save the configuration. It will be applied via **hot-reload** without restarting the container.

---

6. Troubleshooting Forensic Guide

Incident	Frequent Cause	Quick Fix
`PERMISSION_DENIED` on Cloud Storage during build	The account `{PROJECT_NUMBER}-compute@developer.gserviceaccount.com` lacks permissions to write to the Cloud Build bucket.	Run: `gcloud projects add-iam-policy-binding "$PROJECT_ID" --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" --role="roles/storage.admin"`
`auth/unauthorized-domain` when logging in	The Cloud Run URL is not added to the Firebase Authentication Authorized Domains.	Go to **Firebase Console > Authentication > Settings > Authorized domains** and add the service domain.
`Identity Toolkit API has not been used`	The Firebase Auth API was not enabled in the project.	Run: `gcloud services enable identitytoolkit.googleapis.com --project="$PROJECT_ID"`
`Container failed to start and listen on PORT=8080`	Node.js runtime error or missing dependencies in the container.	Check the logs with: `gcloud run services logs read "$SERVICE_NAME" --region "$REGION" --limit 50`
`FAILED_PRECONDITION: One or more users named in the policy do not belong to a permitted customer`	The organization prohibits `allUsers` via organizational policy (`iam.allowedPolicyMemberDomains`).	Assign the role to the corporate domain: `gcloud run services add-iam-policy-binding "$SERVICE_NAME" --member="domain:yourcompany.com" --role="roles/run.invoker" --region="$REGION"`
`PERMISSION_DENIED` on Cloud Firestore	The Firestore database has not been created in Native mode.	Run the command from Step 5: `gcloud firestore databases create --location="nam5" --type="firestore-native"`

---

7. Maintenance, Updates, and Monitoring

Upgrading to a New Version of AegisAI

To deploy patches or new versions of the platform with **Zero-Downtime**:

```
git pull origin main
./deploy_cloud_run.sh "$PROJECT_ID"
```

Cloud Run will spin up the new container, validate its availability through health checks, and automatically route 100% of the traffic to it.